

Tài liệu tổng hợp kiến thức Java cơ bản, với code mẫu cho từng phần. Sử dụng để học và tham khảo.

Lớp Account

Account mô tả tài khoản ngân hàng.

```
public class Account {  
  
    private int id;  
    private String name;  
    private int balance = 0;  
  
    public Account(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    public Account(int id, String name, int balance) {  
        this.id = id;  
        this.name = name;  
        this.balance = balance;  
    }  
  
    public int getId() {  
        return id;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public int getBalance() {  
        return balance;  
    }  
}
```



```

    public void deposit(int amount) {
        balance += amount;
    }

    public void withdraw(int amount) {
        if (balance >= amount) {
            balance -= amount;
        } else {
            System.out.println("That amount exceeds your current
balance.");
        }
    }

    public void display() {
        System.out.println("Id: " + id);
        System.out.println("Name: " + name);
        System.out.println("Balance: " + balance);
    }

    public static void main(String[] args) {
        Account acc = new Account(1, "John", 100);
        acc.deposit(50);
        acc.withdraw(20);
        acc.display(); // Id: 1, Name: John, Balance: 130
    }
}

```

1. Compile, JVM, JRE, JDK

Compile: `javac HelloWorld.java` → bytecode (.class).

Run: `java HelloWorld` → mã máy.

Java platform-independent nhờ JVM.

```

public class HelloWorld {

    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}

```



```
}  
// javac HelloWorld.java  
// java HelloWorld
```

2. Comment

Không compile code comment.

```
public class Comments {  
  
    public static void main(String[] args) {  
        // Single-line  
        System.out.println("Hello");  
  
        /* Multi-line  
           comment */  
        System.out.println("World");  
    }  
}
```

3. Kiểu Dữ Liệu

Primitive: byte, short, int, long, float, double, char, boolean.

Non-primitive: String, array.

Kiểu Dữ Liệu

Kiểu	Kích thước	Default
byte	1 byte	0
short	2 bytes	0
int	4 bytes	0
long	8 bytes	0L
float	4 bytes	0.0f
double	8 bytes	0.0
char	2 bytes	"
boolean	1 bit	false

```
public class DataTypes {

    public static void main(String[] args) {
        byte b = 10;
        int i = 0b1001; // Binary: 9
        int hex = 0x123; // Hex: 291
        String str = "Hello";
        System.out.println(i); // 9
    }
}
```

4. Quy Tắc Đặt Tên Biến

Ý nghĩa, chữ/số/\$, _ (số không đầu), không keyword, camelCase, case-sensitive.

```
public class VariableNaming {

    public static void main(String[] args) {
        int userAge = 25;
        System.out.println(userAge); // 25
    }
}
```

5. In Ra/Nhập Dữ Liệu

println, print, printf.

Nhập: Scanner.

```
import java.util.Scanner;

public class InputOutput {

    public static void main(String[] args) {
        System.out.println("Line");
        System.out.print("No line");
        System.out.printf(" %d\n", 10);

        Scanner sc = new Scanner(System.in);
        int num = sc.nextInt();
    }
}
```



```
        System.out.println(num);
    }
}
```

6. Toán Tử

Arithmetic, comparison (== địa chỉ, equals giá trị).

Logic: &&, || short-circuit.

```
public class Operators {

    public static void main(String[] args) {
        int a = 5, b = 3;
        System.out.println(a + b); // 8
        System.out.println("hi".equals("hi")); // true
        System.out.println(true && false); // false
    }
}
```

7. Final

Hằng số, không thay đổi.

```
public class FinalExample {

    final int NUMBER = 9;

    public void method(final int param) {
        System.out.println(param);
    }

    public static void main(String[] args) {
        FinalExample ex = new FinalExample();
        ex.method(5);
    }
}
```



8. Mảng

Cố định kích thước, index từ 0.

```
import java.util.Arrays;

public class ArraysExample {

    public static void main(String[] args) {
        int[] arr = {1, 2, 3};
        arr[0] = 10;
        int[][] arr2D = new int[2][3];
        System.out.println(Arrays.toString(arr)); // [10, 2, 3]
        System.out.println(Arrays.deepToString(arr2D));
    }
}
```

9. Vòng Lặp

for, enhanced for, while, do-while.

continue, break, labeled.

```
public class Loops {

    public static void main(String[] args) {
        for (int i = 0; i < 5; i += 2) {
            System.out.print(i + " "); // 0 2 4
        }
        int[] arr = {1, 2, 3};
        for (int num : arr) {
            System.out.print(num + " "); // 1 2 3
        }
        outer:
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (j == 1) {
                    break outer;
                }
                System.out.print(j + " "); // 0
            }
        }
    }
}
```



```

    }
}

```

10. Switch Case

Cho int, String, etc.

Default, break.

```

public class SwitchCase {

    public static void main(String[] args) {
        int day = 1;
        switch (day) {
            case 1:
                System.out.println("Monday");
                break;
            default:
                System.out.println("Other");
        }
    }
}

```

11. While/Do While

```

public class WhileLoops {

    public static void main(String[] args) {
        int i = 0;
        while (i < 3) {
            System.out.print(i++ + " "); // 0 1 2
        }
        i = 0;
        do {
            System.out.print(i++ + " ");
        } while (i < 3); // 0 1 2
    }
}

```

12. If/Else

```
public class IfElse {  
  
    public static void main(String[] args) {  
        int num = 5;  
        if (num > 0) {  
            System.out.println("Positive");  
        } else if (num < 0) {  
            System.out.println("Negative");  
        } else {  
            System.out.println("Zero");  
        }  
    }  
}
```

13. Casting

Widening: byte → double.

Up/downcasting.

```
class Parent {  
}  
  
class Child extends Parent {  
}  
  
public class Casting {  
  
    public static void main(String[] args) {  
        int i = 10;  
        double d = i;  
        Parent p = new Child(); // Up  
        Child c = (Child) p; // Down  
    }  
}
```



14. Reference Types

```
public class ReferenceTypes {  
  
    String str; // null  
  
    public static void main(String[] args) {  
        System.out.println(new ReferenceTypes().str); // null  
    }  
}
```

15. Lớp và Đối Tượng

```
class Car {  
  
    String color, owner;  
  
    Car(String color, String owner) {  
        this.color = color;  
        this.owner = owner;  
    }  
}  
  
public class ClassObject {  
  
    public static void main(String[] args) {  
        Car myCar = new Car("black", "Huy");  
        System.out.println(myCar.color); // black  
    }  
}
```

16. Constructor

```
class MyClass {  
  
    int x;  
  
    MyClass() {  
    }  
}
```



```

        MyClass(int x) {
            this.x = x;
        }
    }

    public class Constructors {

        public static void main(String[] args) {
            MyClass obj = new MyClass(5);
            System.out.println(obj.x); // 5
        }
    }

```

17. Biến

Instance, local, static, parameter.

```

class Student {

    String name; // Instance
    static int count; // Static

    void method(int param) {
        int local = 0;
        System.out.println(local + param);
    }
}

public class Variables {

    public static void main(String[] args) {
        Student s = new Student();
        s.name = "Nam";
        Student.count++;
        s.method(5); // 5
    }
}

```



18. Thứ Tự Load

```
class LoadOrder {  
  
    static int staticVar = 1;  
    int instanceVar = 2;  
  
    static {  
        System.out.println("Static");  
    }  
  
    {  
        System.out.println("Instance");  
    }  
}  
  
public class LoadExample {  
  
    public static void main(String[] args) {  
        new LoadOrder(); // Static, Instance  
    }  
}
```

19. String

Immutable, Builder/Buffer mutable.

```
public class Strings {  
  
    public static void main(String[] args) {  
        String str = "Hello";  
        str.concat(" World"); // Không đổi str  
        System.out.println(str); // Hello  
  
        StringBuilder sb = new StringBuilder("Hello");  
        sb.append(" World");  
        System.out.println(sb); // Hello World  
    }  
}
```



20. Wrapper/Autoboxing

```
class Wrappers {

    Integer i = 10; // Autoboxing
    int p = i; // Unboxing

    public static void main(String[] args) {
        Wrappers w = new Wrappers();
        System.out.println(w.p + 10); // 20
    }
}

public class Wrappers {

    public static void main(String[] args) {
        Integer i = 10; // Auto
        int p = i; // Unbox
        System.out.println(p + 10); // 20
    }
}
```

21. this

```
class Employee {

    String name;

    String getName() {
        return this.name;
    }
}

public class ThisKeyword {

    public static void main(String[] args) {
        Employee e = new Employee();
        e.name = "Nam";
        System.out.println(e.getName()); // Nam
    }
}
```



22. Super

```
class Animal {  
  
    String name;  
  
    Animal(String name) {  
        this.name = name;  
    }  
}  
  
class Dog extends Animal {  
  
    Dog(String name) {  
        super(name);  
        System.out.println(super.name);  
    }  
}  
  
public class SuperKeyword {  
  
    public static void main(String[] args) {  
        new Dog("Buddy"); // Buddy  
    }  
}
```

23. Instanceof

```
class Animal {  
}  
  
class Dog extends Animal {  
}  
  
public class InstanceofExample {  
  
    public static void main(String[] args) {  
        Animal a = new Dog();  
    }  
}
```



```
        System.out.println(a instanceof Dog); // true
    }
}
```

24. Encapsulation

```
class Person {

    private String name;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        if (name != null) {
            this.name = name;
        }
    }
}

public class EncapsulationExample {

    public static void main(String[] args) {
        Person p = new Person();
        p.setName("John");
        System.out.println(p.getName()); // John
    }
}
```



7. Abstraction

- **Abstract class:** Có abstract method (no body), con phải override nếu không abstract.
- **Interface:** All abstract, đa kế thừa (implements). No constructor.

Tiêu chí	Abstract Class	Interface
Biến	Non-static, static, final	Static final
Method	Abstract/non-abstract	Abstract
Kế thừa	Extend 1	Implement nhiều
Constructor	Có	Không

Ngoại Lệ

1. Ngoại lệ: Lỗi không mong muốn.

- **Checked:** Từ Exception, báo compile nếu không xử lý.
- **Unchecked:** Từ RuntimeException, không báo.

2. Xử lý: Tránh dừng đột ngột.

- **try-catch:** Xử lý ngay, catch con/cha. Multi-catch: Con trước cha.
- **try-finally:** Luôn chạy finally (đóng resource).
- **throws:** Ném cho method khác, nhiều exception.
- **throw new:** Ném 1.

3. Ném: JVM, API/dev (throw new).

4. Override ngoại lệ:

Ngoại lệ con	Ngoại lệ cha	Cho phép
Unchecked	Checked	OK
Checked	Unchecked	Không
Checked	Checked	Con $\leq$ Cha
Unchecked	Unchecked	OK



Ép Kiểu Widening

byte → short → int → long → float → double

char → int

Kiểu Nguyên Thủy

Kiểu	Kích thước	Giá trị mặc định
byte	1 byte	0
short	2 bytes	0
int	4 bytes	0
long	8 bytes	0L
float	4 bytes	0.0f
double	8 bytes	0.0
char	2 bytes	"
boolean	1 bit	false

25. Inheritance

extends, has-a/is-a.

Loại	Mô tả	Hỗ trợ
Đơn	A extends B	Có
Đa cấp	A extends B extends C	Có
Phân cấp	A,B extends C	Có
Đa	A extends B,C	Không
Lai	Kết hợp	Không

```
class Person {  
  
    String name;  
}
```



```

class Employee extends Person {

    int salary;
}

public class InheritanceExample {

    public static void main(String[] args) {
        Employee e = new Employee();
        e.name = "John";
    }
}

```

26. Polymorphism

Overload, override, up/downcasting.

```

class Shape {

    void draw() {
        System.out.println("Shape");
    }

    void draw(int x) {
        System.out.println("Overload");
    }
}

class Circle extends Shape {

    @Override
    void draw() {
        System.out.println("Circle");
    }
}

public class PolymorphismExample {

    public static void main(String[] args) {
        Shape s = new Circle();
        s.draw(); // Circle
    }
}

```



```

        s.draw(5); // Overload
    }
}

```

27. Abstraction

Abstract class , interface.

Tiêu chí	Abstract Class	Interface
Biến	Non-static/static	Static final
Method	Abstract/non	Abstract
Kế thừa	Extend 1	Implement nhiều
Constructor	Có	Không

```

abstract class Animal {

    abstract void makeSound();
}

interface AnimalEat {

    void eat();
}

class Cat extends Animal implements AnimalEat {

    void makeSound() {
        System.out.println("Meow");
    }

    public void eat() {
        System.out.println("Cat food");
    }
}

public class AbstractionExample {

    public static void main(String[] args) {

```

```

        Cat c = new Cat();
        c.makeSound(); // Meow
    }
}

```

28. Ngoại Lệ

Checked/unchecked.

try-catch, throws.

Con	Cha	Cho phép
Unchecked	Checked	OK
Checked	Unchecked	Không
Checked	Checked	Con ≤ Cha
Unchecked	Unchecked	OK

```

public class Exceptions {

    public static void main(String[] args) {
        try {
            throw new Exception("Error");
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            System.out.println("Finally");
        }
    }
}

```